

Software Requirements: Analysis and Specification

Functional and Non-Functional Requirements

- ▶ Functional requirements describe the functions that the software is to execute;
 - ▶ For example, formatting some text, calculate profit etc.
 - ▶ They are sometimes known as **capabilities or features**.
- ▶ A **functional requirement** can also be described as one for which a finite set of test steps can be written to validate its behavior.

Functional and Non-Functional Requirements

- ▶ Functional requirements describe the functions that the software is to execute;
 - ▶ For example, formatting some text, calculate profit etc.
 - ▶ They are sometimes known as **capabilities or features**.
- ▶ A **functional requirement** can also be described as one for which a finite set of test steps can be written to validate its behavior.
- ▶ Non-functional requirements are the ones that act to constrain the solution.
 - ▶ Non-functional requirements (NFRs) are sometimes known as **constraints or quality attributes / requirements**.
 - ▶ They can be further classified according to whether they are **performance requirements**, **maintainability requirements**, **safety requirements**, **reliability requirements**, **security requirements**, **interoperability requirements** or one of many other types of software requirements.

Requirements Analysis and Specification

Many projects fail:

- ▶ Because they start implementing the system.
- ▶ Without determining whether they are building what the customer really wants.

It is important to learn:

- ▶ Requirements analysis and specification techniques carefully.

Requirements Analysis and Specification

Many projects fail:

- ▶ Because they start implementing the system.
- ▶ Without determining whether they are building what the customer really wants.

It is important to learn:

- ▶ Requirements analysis and specification techniques carefully.

Goals of requirements analysis and specification phase:

- ▶ Fully understand the user requirements.
- ▶ Remove inconsistencies, anomalies, etc. from requirements.
- ▶ Document requirements properly in an SRS document.

Consists of two distinct activities:

- ▶ Requirements Gathering and Analysis
- ▶ Specification

Who Carries Out Requirements Analysis & Specification?

The person who undertakes Reqs. Analysis & Specification:

- ▶ Generally, the systems analyst.
- ▶ Collects data pertaining to the product
- ▶ Analyzes collected data:
 - ▶ To understand what exactly needs to be done.
- ▶ Writes the Software Requirements Specification (SRS) document.

Requirements Analysis and Specification

Final output of this phase:

- ▶ Software Requirements Specification (SRS) Document.

The SRS document is reviewed by the customer.

- ▶ Reviewed SRS document forms the basis of all future development activities.

Requirements Analysis

Requirements analysis consists of two main activities:

- ▶ Requirements gathering
- ▶ Analysis of the gathered requirements

Analyst gathers requirements through:

- ▶ Observation of existing systems,
- ▶ Studying existing procedures,
- ▶ Discussion with the customer and end-users,
- ▶ Analysis of what needs to be done, etc.

Requirements Gathering

Also known as requirements elicitation.

If the project is to automate some existing procedures

- ▶ e.g., automating existing manual accounting activities,
- ▶ The task of the system analyst is a little easier
- ▶ Analyst can immediately obtain:
 - ▶ input and output formats
 - ▶ accurate details of the operational procedures
- ▶ Key Activities
 - ▶ Studying the existing documentation
 - ▶ Interview
 - ▶ Task analysis
 - ▶ Scenario analysis
 - ▶ Form analysis

Requirements Gathering (CONT.)

In the absence of a working system,

- ▶ Lot of imagination and creativity are required.

Interacting with the customer to gather relevant data:

- ▶ Requires a lot of experience.

Some desirable attributes of a good system analyst:

- ▶ Good interaction skills,
- ▶ Imagination and creativity,
- ▶ Experience.

Analysis of the Gathered Requirements

Main purpose of requirements analysis:

- ▶ Clearly understand the user requirements,
- ▶ Detect inconsistencies, ambiguities, and incompleteness.

Incompleteness and inconsistencies:

- ▶ Resolved through further discussions with the end-users and the customers.

Inconsistent Requirement

Some part of the requirement **contradicts** with some other part.
Example:

- ▶ One customer says turn off heater and open water shower when temperature > 100 DegCel
- ▶ Another customer says turn off heater and turn ON cooler when temperature > 100 DegCel

Incomplete Requirement

Some requirements have been **omitted** possibly due to oversight.
Example:

- ▶ Analyst has not recorded what happens when temperature falls below 90 deg cel.
 - ▶ heater should be turned ON
 - ▶ water shower turned OFF.

Analysis of Gathered Requirements (CONT.)

Requirements analysis involves:

- ▶ Obtaining a clear, in-depth understanding of the product to be developed,
- ▶ Remove all ambiguities and inconsistencies from the initial customer perception of the problem.

It is quite difficult to obtain:

- ▶ A clear, in-depth understanding of the problem:
- ▶ Especially if there is no working model of the problem.

Experienced analysts take considerable time:

- ▶ To understand the exact requirements the customer has in his mind.

Analysis of Gathered Requirements (CONT.)

Experienced systems analysts know - often as a result of painful experiences

- ▶ Without a clear understanding of the problem, it is impossible to develop a satisfactory system.

Several things about the project should be clearly understood by the analyst:

- ▶ What is the problem?
- ▶ Why is it important to solve the problem?
- ▶ What are the possible solutions to the problem?
- ▶ What complexities might arise while solving the problem?

Analysis of Gathered Requirements (CONT.)

Some anomalies and inconsistencies can be very subtle:

- ▶ Escape even most experienced eyes.
- ▶ If a formal model of the system is constructed,
- ▶ Many of the subtle anomalies and inconsistencies get detected.

After collecting all data regarding the system to be developed,

- ▶ Remove all inconsistencies and anomalies from the requirements,
- ▶ Systematically organize requirements into a Software Requirements Specification (SRS) document.

Software Requirements Specification (SRS)

Main aim of requirements specification:

- ▶ Systematically organize the requirements arrived during requirements analysis.
- ▶ Document requirements properly.

SRS document is useful in various contexts:

- ▶ Statement of user needs
- ▶ Contract document
- ▶ Reference document
- ▶ Definition for implementation

SRS: A Contract Document

Requirements document is a reference document.

SRS document is a contract between the development team and the customer.

Once SRS document is approved by customer,

- ▶ Any subsequent controversies are settled by referring the SRS document.

Once customer agrees to SRS document:

- ▶ Development team starts to develop (rather prepare a detailed design) the product according to the requirements recorded in the SRS document.

The final product will be acceptable to the customer:

- ▶ As long as it satisfies all the requirements recorded in the SRS document.

About SRS Document

SRS document concentrates on:

- ▶ What needs to be done
- ▶ Carefully avoids the solution (“how to do”) aspects.

The SRS document serves as a contract

- ▶ Between development team and the customer.
- ▶ Should be carefully written

The requirements at this stage:

- ▶ Written using end-user terminology.

If necessary:

- ▶ Later a formal requirement specification may be developed from it.

Properties of a Good SRS Document

It should be **concise**

- ▶ and at the same time should **not be ambiguous**.

It should specify **what the system must do**

- ▶ and **NOT say how to do it**.

Easy to change., i.e. it should be **well-structured**.

It should be **consistent**.

It should be **complete**.

It should be **traceable**.

- ▶ You should be able to trace which part of the specification corresponds to which part of the design, code, etc and vice versa.

It should be **verifiable**.

- ▶ e.g. “system should be user friendly” is not verifiable

Elements of SRS Document

SRS document, normally contains three important parts:

- ▶ Functional requirements,
- ▶ Non-functional requirements,
- ▶ Goals of Implementation.

Non-functional Requirements

Why? - Characteristics of the system which can not be expressed as functions

Nonfunctional requirements include:

- ▶ Reliability issues, Performance issues:
- ▶ **Example:** How fast the system can produce results so that it does not overload another system to which it supplies data?
- ▶ Human-computer interface issues, Interface with other external systems,
- ▶ Security, Portability, Scalability, Usability, Maintainability, etc.
- ▶ Hardware to be used
- ▶ Operating system or DBMS to be used
- ▶ Capabilities of I/O devices
- ▶ Standards compliance
- ▶ Data representations by the interfaced system.....

Brief Comparison

	Functional Reqs.	Non-functional Reqs.
Objective	Describes what the product does	Describes how the product works
End result	Define product features	Define product properties
Focus	Focus on user reqs.	Focus on user expectations
Essentiality	They are mandatory	They are not mandatory but desirable
Origin type	Usually defined by the user	Usually defined by developers or other tech experts
Testing	Component, API, UI testing, etc. Tested before non-functional testing	Performance, usability, security testing, etc. Tested after functional testing
Types	Authentication, auth. levels, data processing, reporting, etc.	Usability, reliability, scalability, performance, etc.

Commonly considered NFRs

- ▶ Performance. How fast does the system return results?
- ▶ Scalability. How much will performance change with higher workloads?
- ▶ Portability. Which hardware, operating systems, and browsers, along with their versions, does the software run on?
- ▶ Compatibility. Does the system conflict with other applications and processes?
- ▶ Reliability. How often does the system experience critical failures?
- ▶ Maintainability. How much time does it take to fix the issue when it arises?
- ▶ Availability. How long is the average system downtime?
- ▶ Security. How well are the system and its data protected against attacks?
- ▶ Usability. How easy is it to use the system?

Example

- ▶ Non-functional requirements specify how the system must perform.
- ▶ For a messaging platform example, a non-functional requirement can be the speed with which a system must perform editing to satisfy user expectations,
- ▶ e.g., “The message must be updated for all users in a chat within 0.1 seconds, given that all users are online and have LTE connection or better.”

Goals of Implementation

Goals describe things that are desirable of the system:

- ▶ But, would **NOT be checked for compliance.**
- ▶ For example, Reusability issues
- ▶ Functionalities to be developed in future

Goals vs. Requirements ?

Examples of Bad SRS Documents

Unstructured Specifications:

- ▶ Narrative essay — one of the worst types of specification document:
- ▶ Difficult to change,
- ▶ Difficult to be precise,
- ▶ Difficult to be unambiguous,
- ▶ Scope for contradictions, etc.

Noise:

- ▶ Presence of text containing information irrelevant to the problem.

Silence:

- ▶ aspects important to proper solution of the problem are omitted.

Examples of Bad SRS Documents

Over-specification:

- ▶ Addressing “how to” aspects
- ▶ For example, “Library member names should be stored in a sorted descending order”
- ▶ Why? - Over-specification restricts the solution space for the designer.

Contradictions:

- ▶ Contradictions might arise
- ▶ if the same thing described at several places in different ways.

Examples of Bad SRS Documents

Ambiguity:

- ▶ Literary expressions
- ▶ Unquantifiable aspects, e.g. “good user interface”, “fantastic processing”, “ultra transfer speed”, etc.

Forward References:

- ▶ References to aspects of problem
- ▶ defined only later on in the text.
- ▶ e.g., R1.1 refers R5.2

Wishful Thinking:

- ▶ Descriptions of aspects
- ▶ for which realistic solutions will be hard to find/achieve.
- ▶ e.g., Successful Search of 1.2 billion adhaar finger-print records in 1 ns.

Structure of Requirements Specifications

For every, functional requirement (with ID), provide the following.

- ▶ Brief Description
- ▶ Sub-requirement (with ID) providing Input and Output
- ▶ Processing
- ▶ Dependency
- ▶ Priority and Risk

When there are too many requirements in large software systems, the software should be split into sections.

- ▶ ERP software
- ▶ Software can also be split into user classes – User stories possible.